Bicol University POLANGUI CAMPUS

Centro Occidental Polangui, Albay

IT112: WEB SYSTEMS and TECHNOLOGIE'S

2nd Semester, AY: 2025-2026

Phase 2: Final Group Project

BentaBoard

"From bawat benta to tunay na kita—track it right."

Proposed By:

PeNiHAMiNo

BSIT 2A

Mantes, Michael B.

Obidece, Hazel Ann R.

Odo, Pearl Anne R.

Racadag, Nica Alyssa R.

Remolar, Noe Jr H.

Presented to:

Dr. Guillermo V. Red, Jr.

IT112 Professor

Introduction

Our Group **"PeNiHaMiNo"** worked together to accomplish *Phase 2: Backend Development and API Integration* for our Final Project in IT112. In this phase, we developed a backend system designed to handle data processing, storage, and communication between the client and server. The project focuses on building a functional and organized server-side application using modern web development technologies.

The **backend was implemented** using Node.js and Express.js, which allowed us to create and manage API endpoints efficiently. For data storage, we used **MongoDB**, with **Mongoose** to define schemas and simplify database interactions.

Throughout this activity, we applied key backend development concepts such as creating RESTful APIs, implementing CRUD (Create, Read, Update, Delete) operations, and organizing code into modular components like **routes**, **controllers**, and **models**. We also tested our API endpoints using tools like Postman to ensure that the system functions correctly.

This laboratory (phase) activity helped us understand how a backend system works in a real-world application and how to structure code in a scalable and maintainable way.

Objectives

At the end of this activity, we were able to:

1. Set up a working backend server using Node.js and Express.js.
2. Connect the backend to MongoDB using Mongoose.
3. Create REST API routes for CRUD operations.
4. Test backed endpoints using Postman/Thunder Client.
5. Organize backend files into a modular folder structure.

Backend Folder Structure

Bash

```
/backend
 ├── server.js
 ├── /routes
├── /models
├── /controllers
 ├── /config
└── .env
```

I. Overview of Backend Setup

Our backend system is developed using **Node.js** and **Express.js** to handle server-side operations and API requests.

The project follows an organized structure. The controllers manage the logic of the application, such as handling user authentication, product management, sales, and expenses. The routes define the API endpoints that connect client requests to the appropriate controller functions. The models represent the structure of the data stored in the database.

We use **MongoDB** as the database, and Mongoose to define schemas and interact with the database efficiently.

The backend folder is structured into:

- controllers/ – contains business logic (e.g., auth, user, product, sales)
- routes/ – handles API endpoints
- models/ – defines database schemas
- middleware/ – handles authentication and request validation
- config/ – stores configuration settings such as database connection

This setup ensures modularity, making the system easier to maintain, scale, and debug.

II. Code Snippets of server.js, Sample Model, **and** Routes

- **Server.js**

```
// Serve frontend
app.use(express.static(path.join(__dirname, "../frontend")));

// API Routes
app.use("/api/users", usersRouter);
app.use("/api/products", productRouter);
app.use("/api/sales", saleRouter);
app.use("/api/expenses", expenseRouter);
app.use("/api/purchases", purchaseRouter);

// Test route
app.get("/api/health", (req, res) =>
  res.json({ status: "ok", message: "BentaBoard API is active" })
);

// Error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: "Something went wrong on the server!" });
});
```

```
// Start server
async function start() {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected");

    const PORT = process.env.PORT || 3000;
    app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
  } catch (err) {
    console.error(err.message);
    process.exit(1);
  }
}

start();
```

This file **initializes the backend server** using Express.js. It configures middleware such as CORS and JSON parsing, **connects to MongoDB** using Mongoose, and links

different route files for handling API endpoints like users, products, sales, and expenses. It also includes a test route and global error handling for debugging.

2. Sample models/User.js

backend > models > User.js > ...

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true
  },
  full_name: {
    type: String,
    required: true
  },
  role: {
    type: String,
    required: true,
    default: 'Seller'
  },
  created_at: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('User', UserSchema);
```

This model defines the structure of user data in the database. It includes fields such as username, password, full name, and role. Validation rules like `required` and `unique` ensure data integrity, while default values (e.g., role and created date) are automatically assigned.

3. routes/userRoutes.js

backend > routes > userRoutes.js > ...

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

// Register
router.post('/register', async (req, res) => {
  try {
    const { username, password, full_name, role } = req.body;

    let user = await User.findOne({ username });
    if (user) return res.status(400).json({ msg: "User already exists" });

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const newUser = new User({
      username,
      password: hashedPassword,
      full_name,
      role
    });
```

```
    await newUser.save();
    res.status(201).json({ msg: "User registered successfully!" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Login
router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;

    const user = await User.findOne({ username });
    if (!user) return res.status(400).json({ msg: "Invalid Credentials" });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ msg: "Invalid Credentials" });

    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '1d' }
    );

    res.json({ token, user: { id: user._id, username: user.username, role: user.role } });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

This **route** file handles user authentication. The register route creates a new user and securely hashes the password using **bcrypt**. The login route verifies user credentials and generates a JSON Web Token (JWT) for authentication. These routes allow secure access to the system.

III. API Testing Screenshots

After creating the backend routes, we tested them using Postman/Thunder Client to make sure they were working correctly.

Routes Tested

- **POST /api/users** - create a new user
- **GET /api/users** - retrieve all users
- **PUT /api/users/:id** - update a user
- **DELETE /api/users/:id** - delete a user

Based on our testing, the API routes worked as expected. The POST request successfully added a user to the database, the GET request displayed the saved users, the PUT request updated the selected user's information, and the DELETE request removed the selected user.

Figure 1. POST /api/users/register

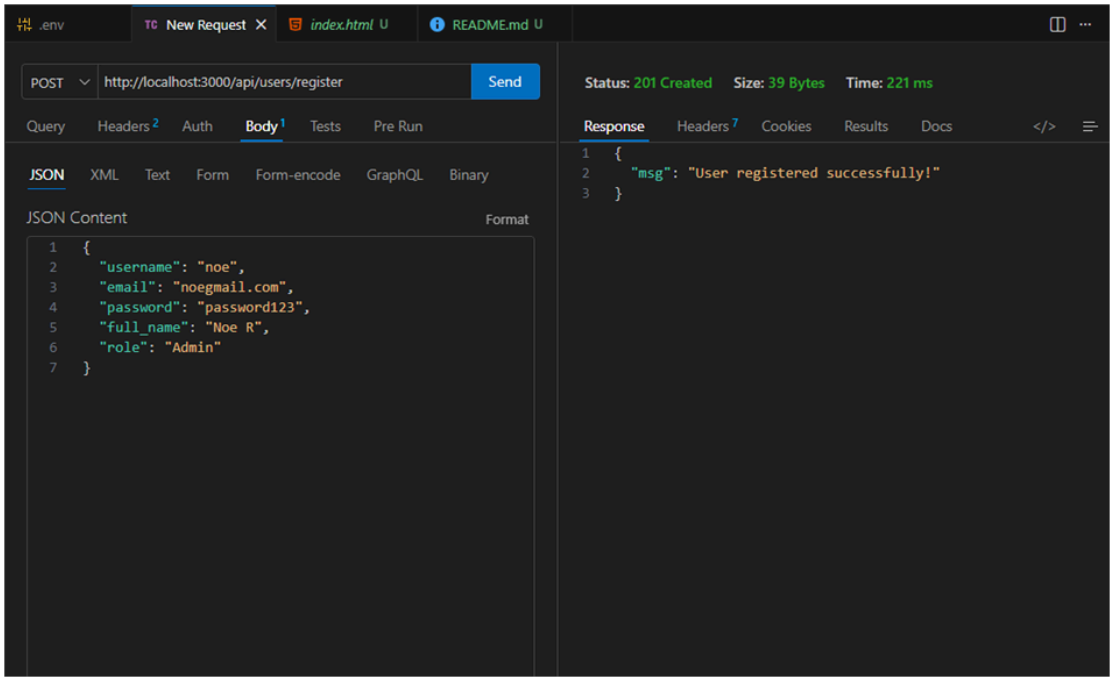


Figure 2. POST /api/users/login

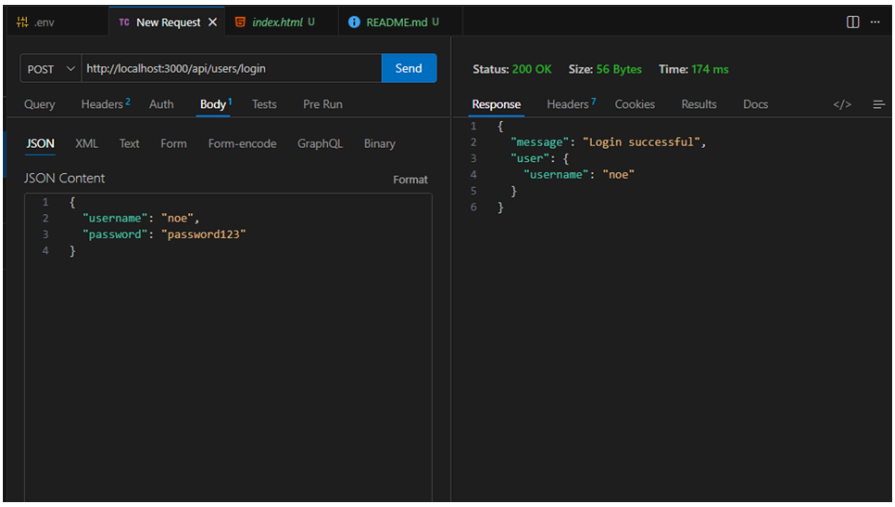


Figure 3. GET /api/users/

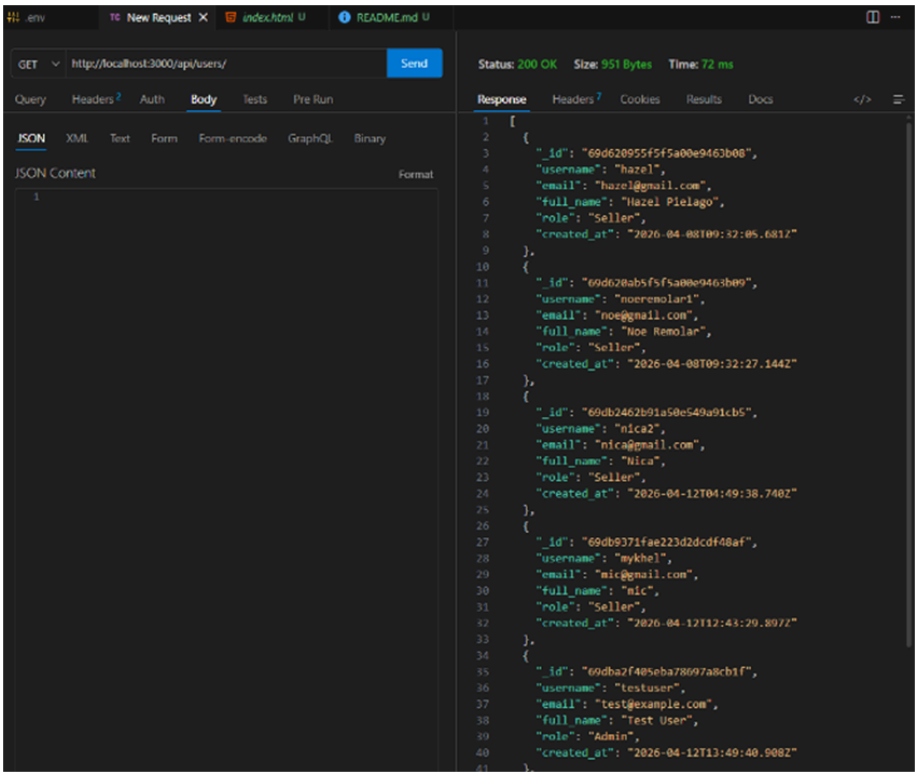


Figure 4. PUT /api/users/ :id

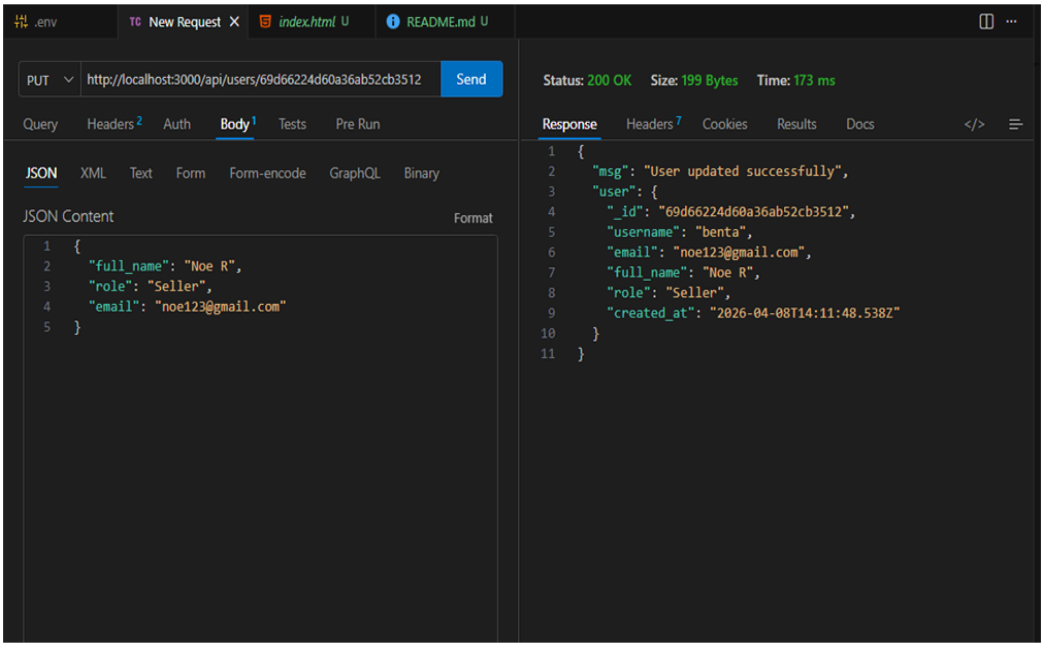


Figure 5. DELETE /api/users/ :id

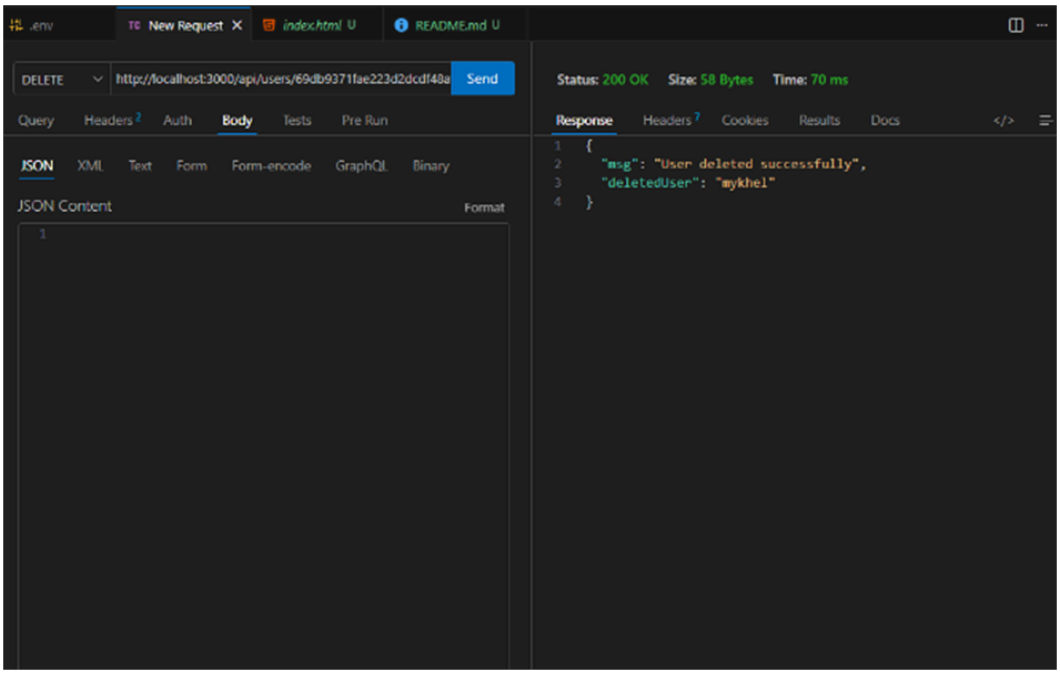


Figure 6. POST /api/users/login

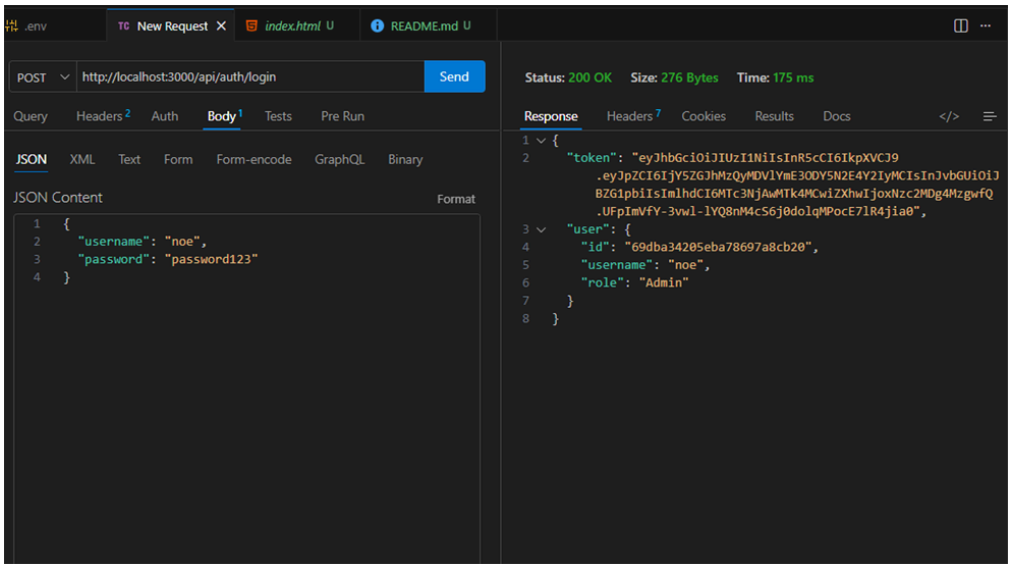


Figure 7. GET /api/products/

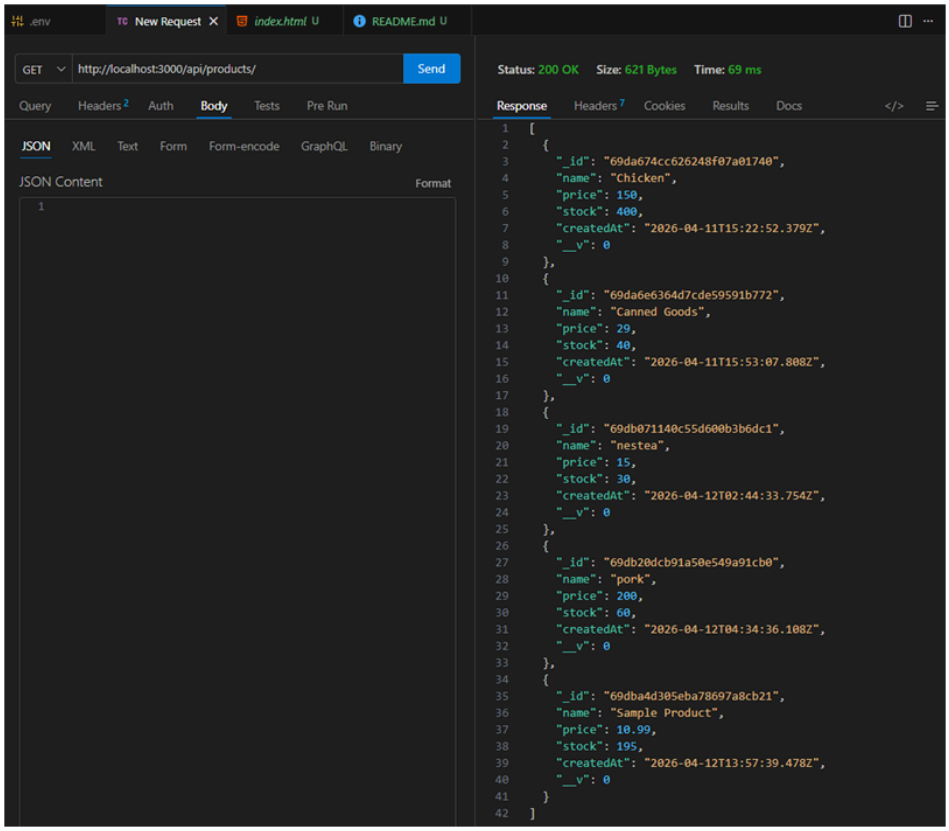


Figure 8. POST /api/products/

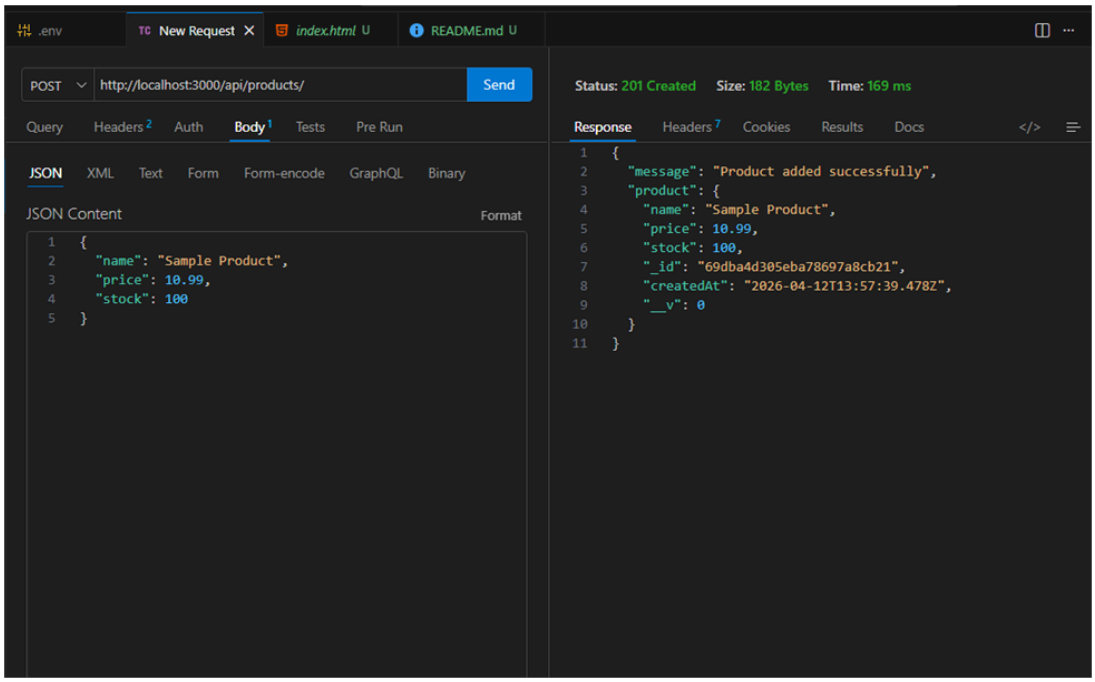


Figure 9. POST /api/sales/

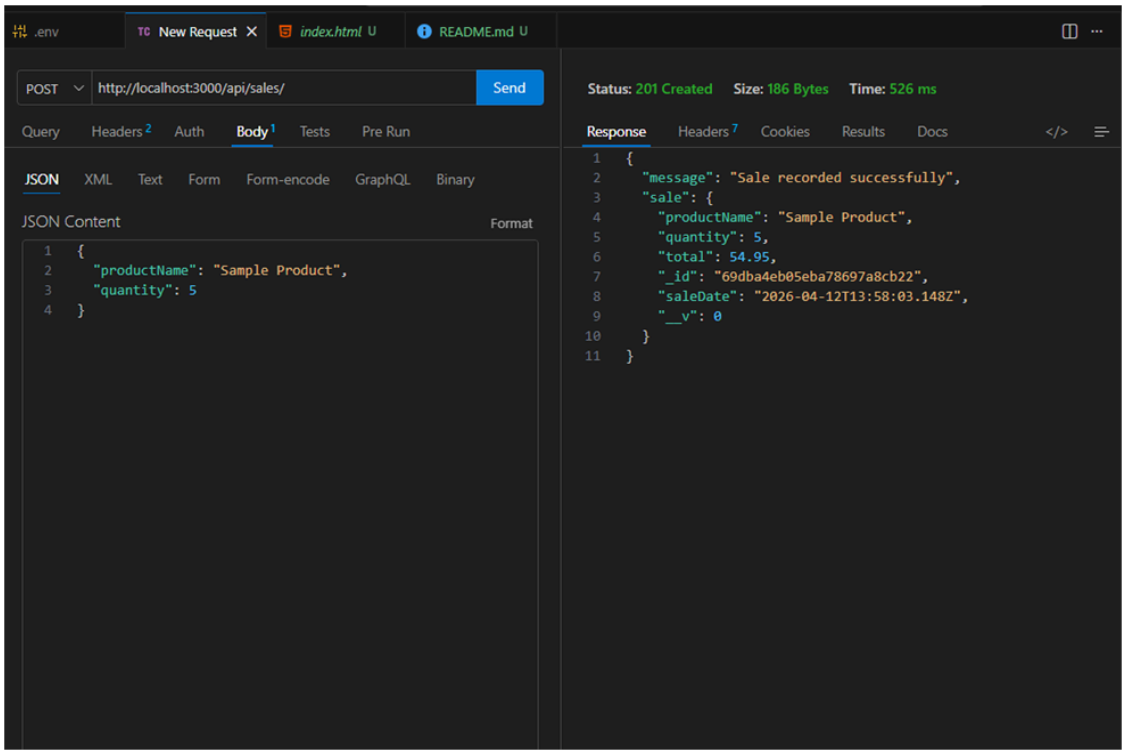


Figure 10. POST /api/expenses/

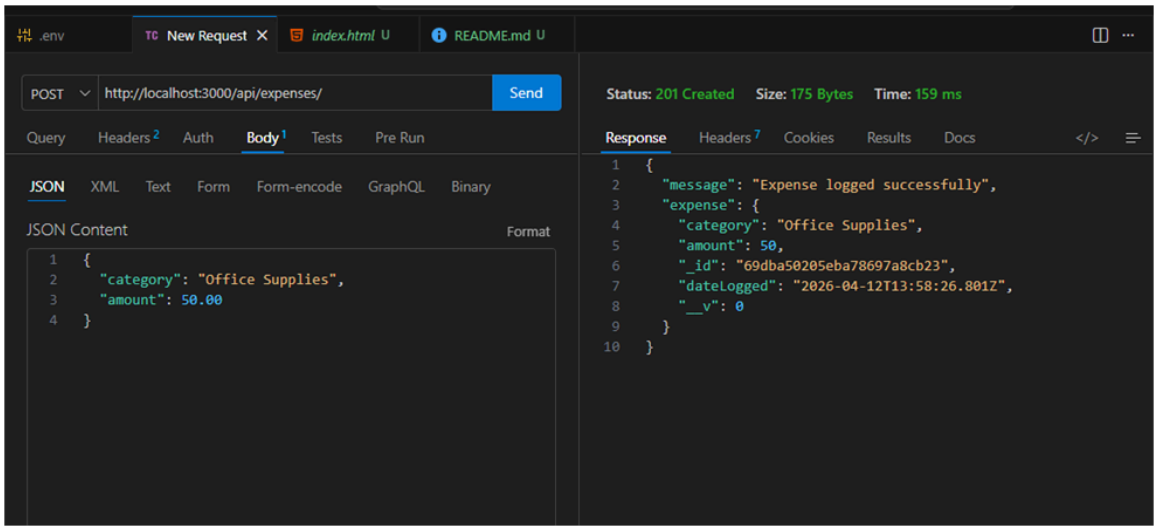


Figure 11. GET /api/purchases/

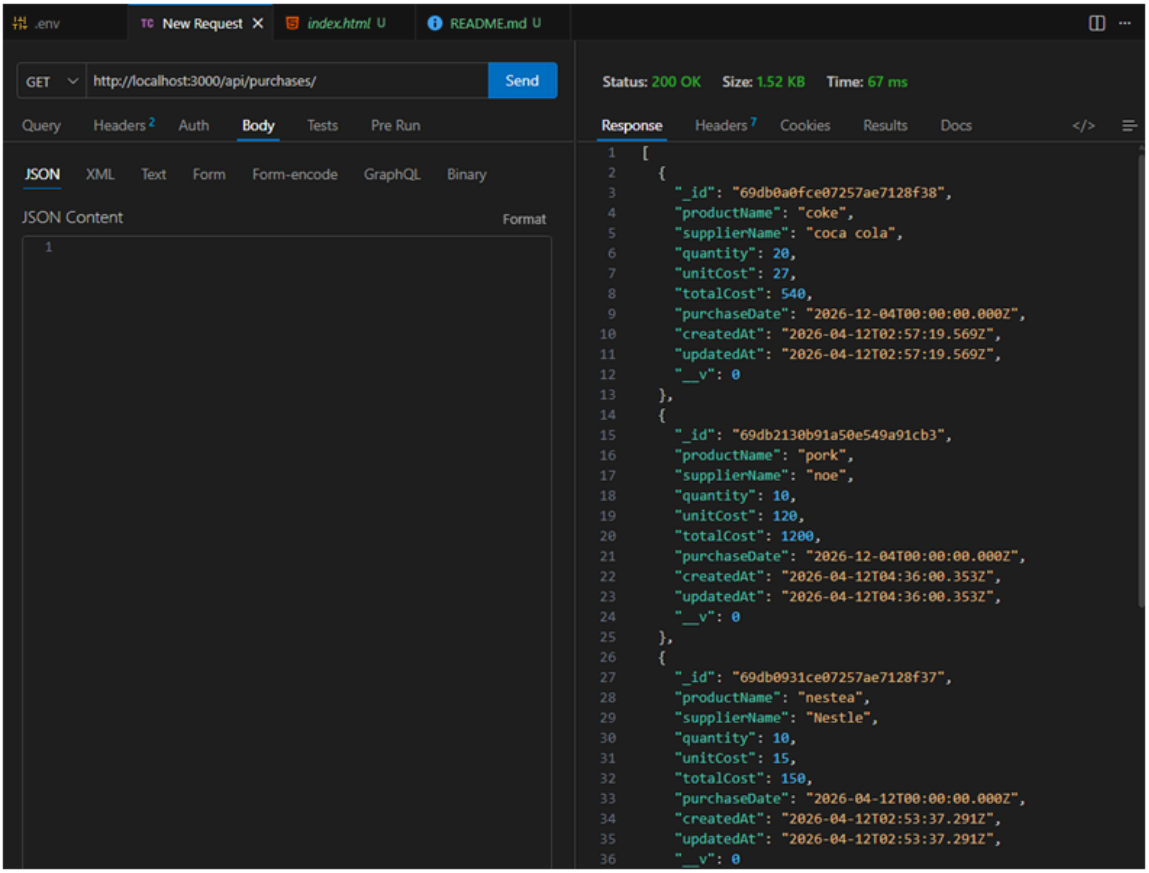


Figure 12. POST /api/expenses/add

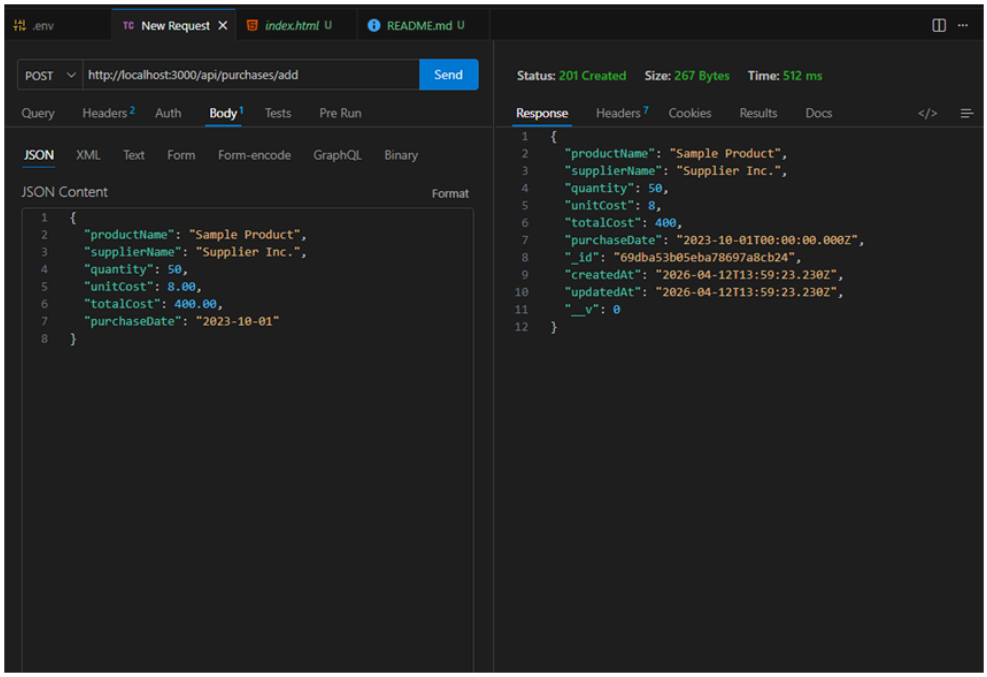
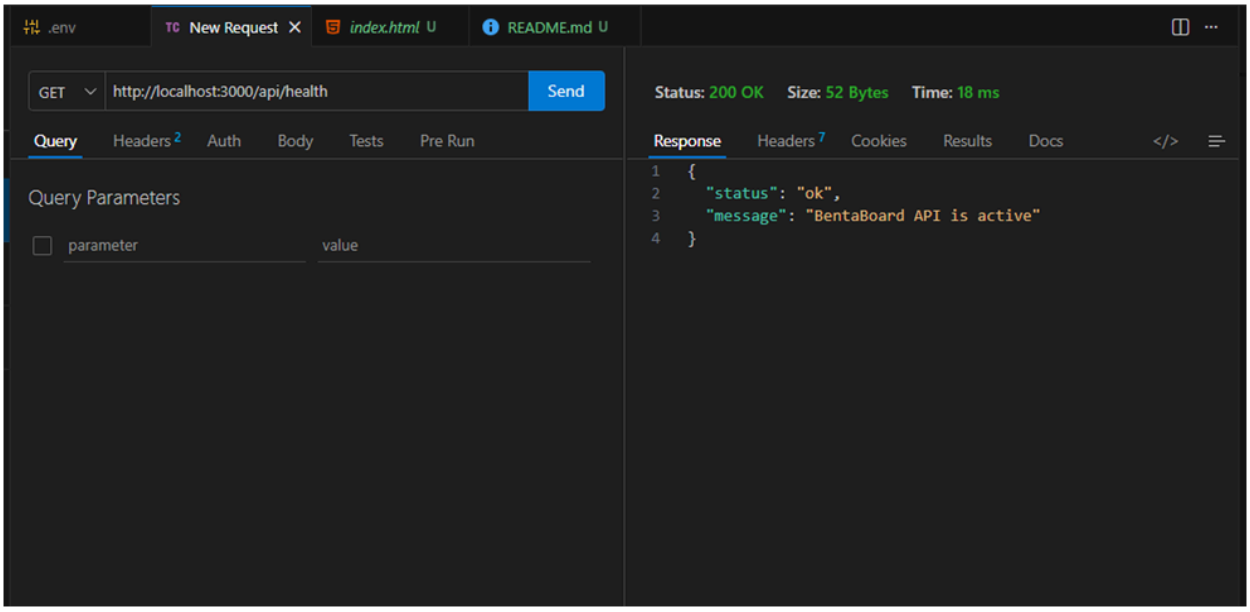


Figure 13. GET /api/health



GitHub Integration:

All backend-related files were properly committed and pushed to the GitHub repository, specifically organized within the `/backend` folder. Each commit was written with clear and meaningful messages to accurately reflect the changes and progress made during this phase of development. This practice ensures better version control, traceability, and collaboration among team members.

In addition, a `README.md` file was created inside the backend folder to serve as documentation for the project. The README provides essential information, including the list of implemented API routes, defined data models, database connection setup, and step-by-step instructions on how to run the backend locally. This documentation helps other developers or users easily understand, set up, and interact with the system.

Figure 13: GitHub Commits History

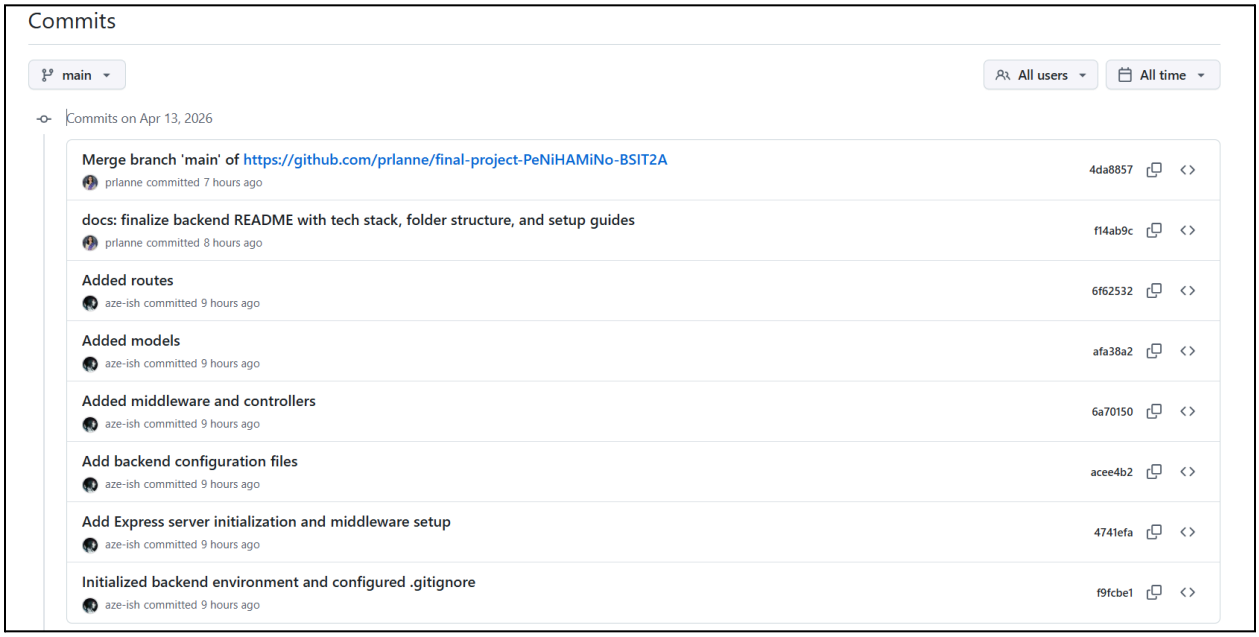


Figure 14: GitHub Backend Commits

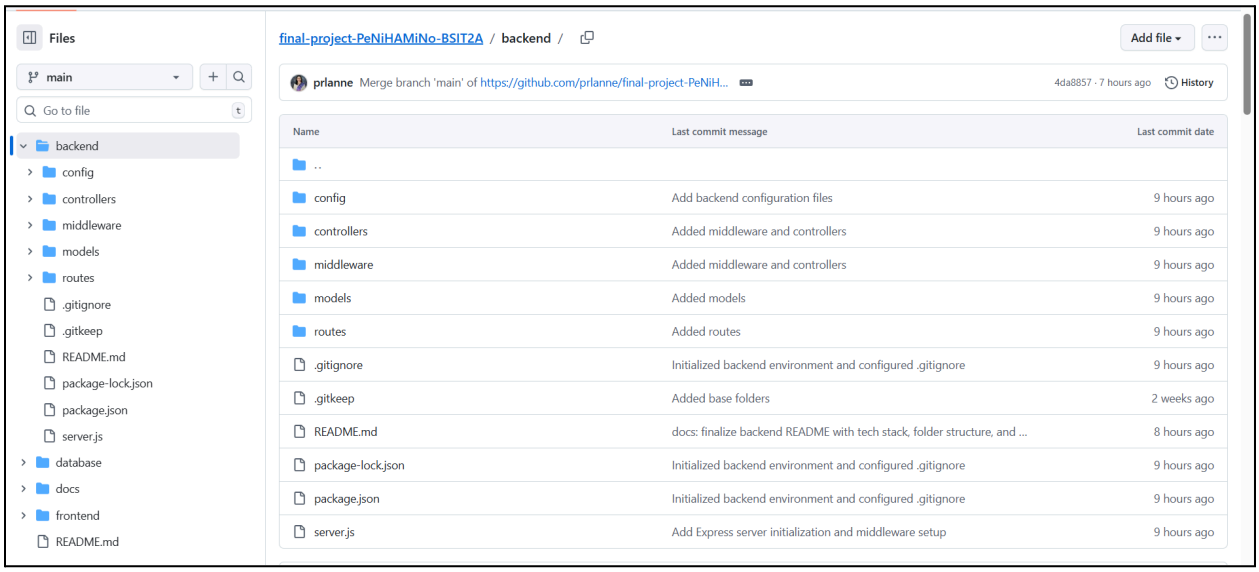
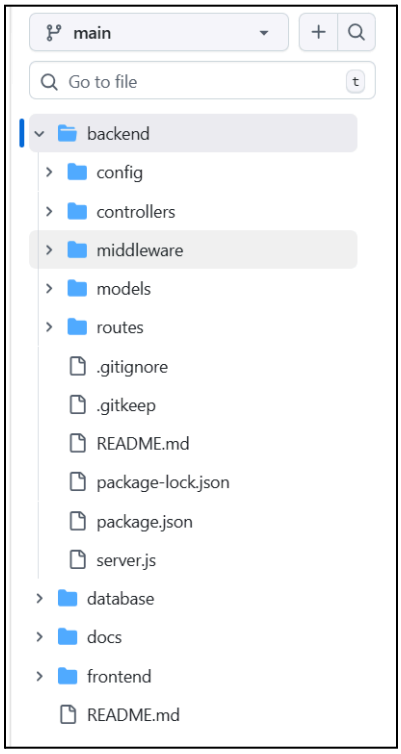


Figure 15. Backend Folder in GitHub repository



IV. Brief explanation of each group member's contribution

Hazel Ann Obidece - Project Manager

I took the lead in setting up the initial backend structure for our project. I created the main folders and organized the files so everything stays neat and easy to find. I also handled the installation of the necessary packages like Express and Dotenv. My work provided the foundation that the rest of the team used to build our server.

Noe Jr Remolar - Documentation Officer

I focused on building the API endpoints that allow our application to function. I wrote the code for the routes so that we can send and receive data easily. I made sure that the Create, Read, Update, and Delete operations were working correctly. This part of the project is what connects our data to the user interface.

Michael Mantes - Database Manager

I was responsible for managing our database and setting up the Mongoose models. I connected our application to MongoDB so that all our information is saved securely. I also defined how our data, like user details or products, should look in the system. This ensures that our backend can store and retrieve information without any errors.

Nica Alyssa Racadag - FrontEnd Manager

I handled the API testing using Postman to make sure all our routes were working. I sent different requests to the server to check if the data was being processed right. By finding and fixing small bugs early, I helped make our backend much more stable. My testing was crucial to ensure that the server doesn't crash during use.

Pear Anne Odo - GitHub Manager

I contributed during this specific phase of the project by writing simple explanations of how each route works so we can understand it later. My work helps keep a clear record of everything our team has accomplished. In addition, I monitored our group's GitHub repository to ensure that updates and changes were properly tracked and organized. I also added clear descriptions to the README.md file for the backend, making it easier for the team to understand the system setup and functionality..